



ORACLE

Academy

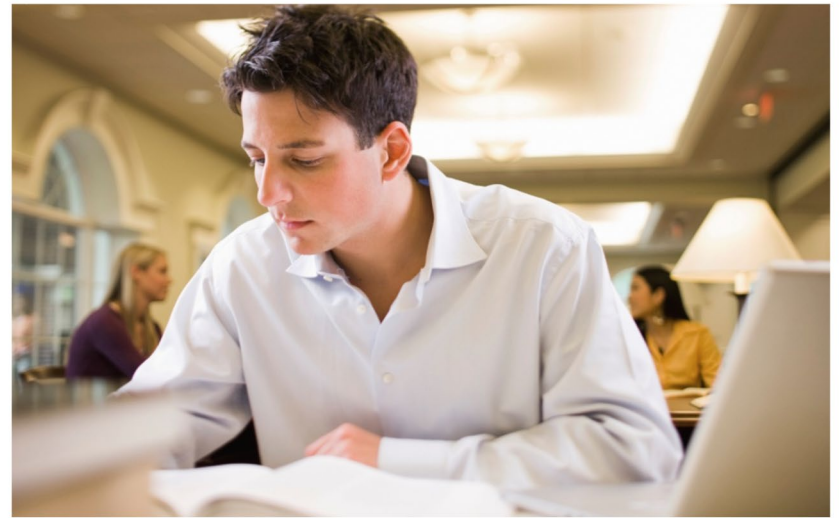


Java Fundamentals

4-3

Opérateurs et types de données

ORACLE
Academy



Objectifs

- Voici les objectifs de cette leçon :
 - Utiliser les types de données primitifs dans le code Java
 - Spécifier des littéraux pour les types primitifs et pour les chaînes
 - Montrer l'initialisation des variables
 - Décrire les règles de portée d'une méthode
 - Déterminer si une expression exige une conversion de type



Présentation

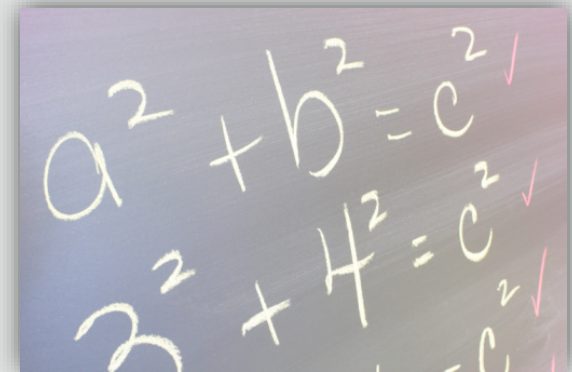
- Cette leçon traite des sujets suivants :
 - Appliquer la conversion (cast) de type dans le code Java
 - Utiliser les opérateurs arithmétiques
 - Utiliser l'opérateur d'affectation
 - Utiliser une méthode de la classe Math
 - Accéder à une méthode de la classe Math à partir de l'API Java

Types de programmation Java

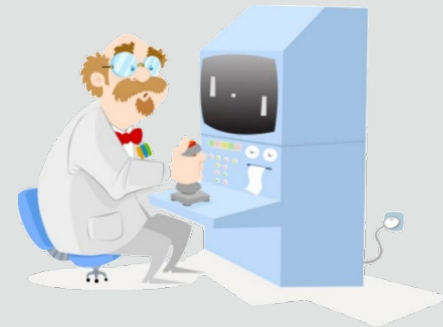
- Caractéristiques des types de données dans Java :
 - Permettent de définir le type de données qui peuvent être stockées dans une variable
 - Garantissent que seules les données appropriées sont stockées
 - Sont déclarés ou déduits
 - Peuvent être créés par le programmeur

Les variables doivent avoir des types de données

- Toutes les variables doivent avoir un type de données pour des raisons de sécurité
- Le programme n'est pas compilé si l'utilisateur essaie de stocker des données dont le type n'est pas correct
- Les programmes doivent respecter les contraintes de type pour être exécutés
 - Les types incorrects dans les expressions ou dans les données sont signalés comme des erreurs au moment de la compilation



Types de données primitifs



- Java propose huit types de données primitifs utilisés pour stocker des données pendant le fonctionnement d'un programme
- Les types de données primitifs sont un groupe spécial de types de données qui n'utilisent pas le mot-clé new lorsqu'ils sont initialisés
- Java les crée en tant que variables automatiques qui ne sont pas des références, qui sont stockées dans la mémoire avec le nom de la variable
- Les types primitifs les plus courants utilisés dans ce cours sont int (entiers) et double (décimaux)

Types de données primitifs



Type de données	Taille	Données d'exemple	Description des données
boolean	1 bit	true, false	Stocke des indicateurs True et False
byte	1 octet (8 bits)	12, 128	Stocke des entiers compris entre -128 et 127
char	2 octets	'A', '5', '#'	Stocke un caractère Unicode 16 bits entre 0 et 65 535
short	2 octets	6, -14, 2 345	Stocke des entiers compris entre -32 768 et 32 767

Types de données primitifs



Type de données	Taille	Données d'exemple	Description des données
int	4 octets	6, -14, 2 345	Stocke des entiers compris entre : -2 147 483 648 et 2 147 483 647
long	8 octets	3 459 111, 2	Stocke des entiers compris entre : -9 223 372 036 854 775 808 et 9 223 372 036 854 775 807
float	4 octets	3,145, 0,077	Stocke un nombre décimal positif ou négatif compris entre : $1,4023 \times 10^{-45}$ et $3,4028 \times 10^{+38}$
double	8 octets	0,0000456, 3,7	Stocke un nombre décimal positif ou négatif compris entre : $4,9406 \times 10^{-324}$ et $1,7977 \times 10^{308}$

Déclaration de variables et utilisation de littéraux

- Le mot-clé `new` n'est pas utilisé lors de l'initialisation d'une variable de type primitif
- A la place, une valeur littérale doit être affectée à chaque variable lors de l'initialisation
- Un littéral peut être un nombre, du texte ou tout autre élément représentant une valeur
- Exemples de déclaration de variable et d'affectation d'une valeur littérale :

```
boolean result = true;  
char capitalC = 'C';  
byte b = 100;  
short s = 10000;  
int i = 100000;
```

Exemple de déclaration de variables et d'utilisation de littéraux

- Les valeurs d1 et d2 sont identiques
- L'initialisation de d2 montre comment utiliser la notation scientifique pour définir la valeur
- La même valeur est affectée à total et à ss_num
- L'initialisation de ss_num montre que des traits de soulignement peuvent être utilisés pour séparer les nombres à des fins de lisibilité

```
long total=999999999;  
long ss_num = 999_99_9999;  
double d1 = 123.4;  
double d2 = 1.234e2;  
float f1 = 123.4f;
```

Exemples de littéraux numériques

- 0x et 0b désignent une valeur hexadécimale littérale ou une valeur binaire littérale
- Les littéraux permettent d'améliorer les performances de traitement

```
long creditCardNumber = 1234_5678_9012_3456L;  
long socialSecurityNumber = 999_99_9999L;  
float pi = 3.14_15F;  
long hexBytes = 0xFF_EC_DE_5E;  
long hexWords = 0xCAFE_BABE;  
long maxLong = 0x7fff_fff_fff_ffffL;  
byte nybbles = 0b0010_0101;  
long bytes = 0b11010010_01101001_10010100_10010010;
```



Littéraux binaires

- Les littéraux binaires peuvent être exprimés à l'aide du système binaire en ajoutant le préfixe 0b ou 0B au nombre
- Les littéraux binaires sont des valeurs Java de type int
- Les valeurs Java de type short et byte doivent être converties afin d'éviter une erreur de perte de précision du compilateur

La conversion consiste à passer d'un type à un autre. Vous convertissez explicitement l'élément de son type d'origine au type que vous spécifiez. Lors d'une conversion de type, un nombre décimal n'est pas arrondi, mais tronqué. Exemple de conversion d'une valeur de type double en int :



```
double x = 5.745;  
int y = (int)x; //y est désormais égal à 5
```

Rôle des littéraux binaires Java

- Usages des littéraux binaires :
 - Calculs
 - Comparaisons
 - Programmation de base, par exemple :
 - Ecriture de pilotes de dispositif
 - Eléments graphiques de base
 - Assemblage de paquets de protocole de communication
 - Décodage

Pourquoi utiliser des littéraux binaires ?

- L'utilisation de littéraux binaires pour représenter des valeurs à comparer ou à calculer est bien plus rapide que l'utilisation de valeurs du type de données réel
- Les processeurs hautes performances modernes effectuent généralement des calculs sur les entiers aussi vite que les littéraux binaires, alors pourquoi utiliser ces derniers ?
- Les littéraux restent à privilégier par rapport aux performances globales, car ils utilisent moins de ressources

Exemple de conversion (cast) de type

```
//Valeur 'byte' 8 bits :  
byte aByte = (byte)0b00100001;  
  
//Valeur 'short' 16 bits :  
short aShort = (short)0b1010_0001_0100_0101;  
  
//Valeurs 'int' 32 bits :  
int anInt1=0b1010_0001_0100_0101_1010_0001_0100_0101;  
int anInt2=0b101;  
int anInt3=0B101;    //La lettre B peut être une  
                     //majuscule ou une minuscule
```

Voici un exemple de conversion de type
Dans cet exemple, la valeur binaire est convertie au type byte

Règles relatives aux noms de variable

- Vous devez respecter les règles suivantes lorsque vous choisissez le nom d'une variable :
 - N'utilisez pas de mot réservé ou mot-clé Java
 - N'utilisez pas d'espace dans le nom de la variable
 - Utilisez une combinaison de lettres, ou une combinaison de lettres et de chiffres
 - Ne commencez pas par un chiffre
 - Les seuls symboles autorisés sont le trait de soulignement (_) et le signe dollar (\$)



Conventions relatives aux noms de variable

- Bien qu'elles ne soient pas des règles, la plupart des programmeurs Java respectent les conventions suivantes :
 - Utiliser des mots complets plutôt que des abréviations non claires
 - Ne pas utiliser des variables d'une seule lettre
 - Si toutes les variables ne se composent que d'une lettre, le code risque de porter à confusion
 - Les variables de contrôle de boucle sont une exception à cette convention, avec les lettres i, j et k
 - Si le nom d'une variable se compose d'un seul mot, toutes les lettres de ce mot doivent être en minuscules
 - Si le nom d'une variable se compose de plusieurs mots, utiliser la notation lowerCamelCase

Autres conventions de dénomination pour les noms de variable

- Autres conventions de dénomination :
 - Si une variable est une valeur constante, toutes les lettres doivent être en majuscules et les mots doivent être séparés par des traits de soulignement
 - Utiliser des noms qui indiquent le rôle de la variable
 - Dans l'exemple ci-dessous, PI est un bon choix de nom pour ce nombre, car il vous rappelle la nature de la variable

```
double PI = 3.14159;
```

Portée d'une variable

- La portée permet de décrire le bloc de code dans lequel une variable existe dans un programme
- Plusieurs variables peuvent avoir le même nom dans un programme Java
 - Dans la plupart des cas, la variable la plus interne est prioritaire
- Une variable existe uniquement dans le bloc de code dans lequel elle a été déclarée
- Une fois que l'accolade finale du bloc (}) est atteinte :
 - La variable est hors de portée
 - Elle n'est plus reconnue comme variable déclarée

Portée de variable - Exemple 1

- Dans l'exemple suivant, le nom ne sera pas affiché car il cesse d'exister une fois que l'accolade marquée Point B est atteinte

```
public void someMethod()  
{  
    if(gameOver && score>highScore)  
    {  
        String name;                //Point A  
        System.out.println("Saisissez votre nom :");  
        name=reader.next();  
    }//fin if                        //Point B  
  
    System.out.println("Merci " + name + ", ");  
    System.out.println("votre meilleur score a été "  
                        + "enregistré.");  
}//fin methode someMethod
```

Portée de variable - Exemple 2

- Dans cet exemple, le nom de la variable a été déplacé en dehors du bloc d'instruction if pour que le nom puisse être utilisé tout au long de la méthode

```
public void someMethod()  
{  
    String name; //Point A  
    if(gameOver && score>highScore)  
    {  
        System.out.println("Saisissez votre nom :");  
        name=reader.next();  
    } //fin if //Point B  
    System.out.println("Merci " + name + ", ");  
    System.out.println("votre meilleur score a été "  
        + "enregistré.");  
} //fin methode someMethod
```

Portée de variable - Exemple 3

- Java autorise l'existence d'une variable de classe et d'une variable de méthode portant le même nom dans un programme
- Pouvez-vous prévoir ce qui sera affiché par ce programme ?

```
public class Counting{
    public static int counter=5;
    public static void main(String[] args){
        System.out.println("Au début de ce programme, le compteur indique " + counter);
        count();
        System.out.println("A la fin de ce programme, le compteur indique "+ counter);
    }//fin methode main

    public static void count(){
        int counter=10;
        System.out.println("A la fin de cette méthode, le compteur indique "+ counter);
    }//fin methode count

} //fin classe Counting
```

Portée de variable - Solution de l'exemple 3

- Solution :
 - 5, 10, 5
- Le programme démarre `main()` et affiche le compteur de variable de classe globale
- La méthode `count()` est appelée et un compteur de variable locale est créé pour cet appel de méthode
- La valeur de 10 lui est donnée, puis apparaît
- Lorsque l'accolade à la fin de `count()` est atteinte, la variable locale devient hors de portée (cesse d'exister)
- Le programme revient à la méthode `main()` et affiche le compteur de variable globale, dont la valeur n'a pas changé

Opérateurs booléens

- Les opérateurs booléens sont un ensemble d'opérateurs permettant de comparer des expressions à True ou False

Opérateur	Opérateur	Description	Exemple
&&	ET	Si les deux sont vrais, renvoie vrai	(A&&B)
	OU	Si certains sont vrais, renvoie vrai	(A B)
!	NE PAS	Inverse l'état logique (T to F, F to T).	!(A&&B)
==	égal à	Si les deux sont égaux, renvoie vrai	(A==B)
!=	pas égal à	Si les deux sont différents, renvoie vrai	(A!=B)
>	plus grand que	Si la valeur de gauche est supérieure à celle de droite, renvoie vrai	(A>B)
>=	plus grand ou égal à	Si la gauche est supérieure ou égale à la droite, renvoie la valeur vrai	(A>=B)
<	moins que	Si la gauche est inférieure à la droite, renvoie vrai	(A<B)
<=	inférieur ou égal à	Si la gauche est inférieure ou égale à la droite, renvoie la valeur vrai	(A<=B)

Opérateurs arithmétiques

- Java applique plusieurs opérateurs arithmétiques pour effectuer des opérations mathématiques



Symbole	Description de l'opérateur
+	Opérateur d'addition
-	Opérateur de soustraction
*	Opérateur de multiplication
/	Opérateur de division (obtient le quotient)
%	Opérateur modulaire (obtient le reste)
++	Opérateur d'incrément (ajoute une unité) Opérateur unaire
--	Opérateur de décrément (soustrait une unité) Opérateur unaire

Priorité des opérateurs arithmétiques

- Toutes les expressions mathématiques sont évaluées par ordre de priorité :
 - Les expressions entre parenthèses sont traitées en premier
 - Ensuite, toutes les multiplications, les divisions et les opérations modulaires sont traitées de gauche à droite
 - Enfin, toutes les additions et les soustractions sont traitées de gauche à droite

Incréments et décréments

- Les incréments et décréments sont traités en premier pour la notation pré-incrément et en dernier pour la notation post-incrément
- Dans Java, un incrément consiste à ajouter une unité à la valeur de la variable
- Dans Java, un décrément consiste à soustraire une unité de la variable
- Notation pré-incrément :

```
++x;
```

- Notation post-incrément :

```
x++;
```



Exemple de priorité pour les incréments et les décréments

- Notation pré-incrément :

```
int x = 3;  
++x;           //x est égal à 4  
z = ++x;       //x est égal à 5, donc z est égal à 5
```

- Notation post-incrément :

```
int x = 3;  
x++;           //x est égal à 4  
z = x++;       //z est égal à 4, donc x est égal à 5
```

Opérateur d'affectation

- Java utilise le signe égal (=) en tant qu'opérateur d'affectation L'évaluation de l'expression de droite est affectée à l'emplacement de mémoire de gauche

```
int x = 4;  
int y = 5;  
int z = 10;  
int total = 12;
```



Opérateur d'affectation - Exemple 1

- Lorsque cette ligne de code est exécutée, la valeur du total change
- Les cases indiquent ce qui se trouve à chaque emplacement de mémoire pour X, Y, Z et le total
- La valeur affectée à l'emplacement de mémoire total est $4 + 5 * 10$, soit 54

```
int x = 4, y = 5, z = 10;  
int total = x + y * z;
```



Opérateur d'affectation - Exemple 2

- Considérez l'opérateur d'affectation comme une flèche pointant vers la gauche
- Tous les éléments qui se trouvent à droite iront dans l'emplacement de mémoire de gauche
- Comment la mémoire sera-t-elle modifiée après l'exécution de ce code ?

```
int x = 4, y = 5, z = 10;  
int total = x + y * (z - x);
```



Opérateur d'affectation - Solution de l'exemple 2

- La réponse est que la valeur de l'expression sera affectée au total
- Cela signifie que la valeur de l'expression sera stockée dans l'adresse de mémoire associée à la variable total

```
int x = 4, y = 5, z = 10;  
int total = x + y * (z - x);
```



Troncature et division d'entiers

- La division de deux entiers génère TOUJOURS un entier
- Par exemple, la formule de géométrie permettant de calculer le volume d'un cône est la suivante :
 - $V = \frac{1}{3} r^2 h$
 - Si $\frac{1}{3}$ est utilisé dans une expression Java, sa valeur est évaluée à 0 en raison de la division d'entiers
 - Le résultat d'une division d'entiers est le quotient sans décimale

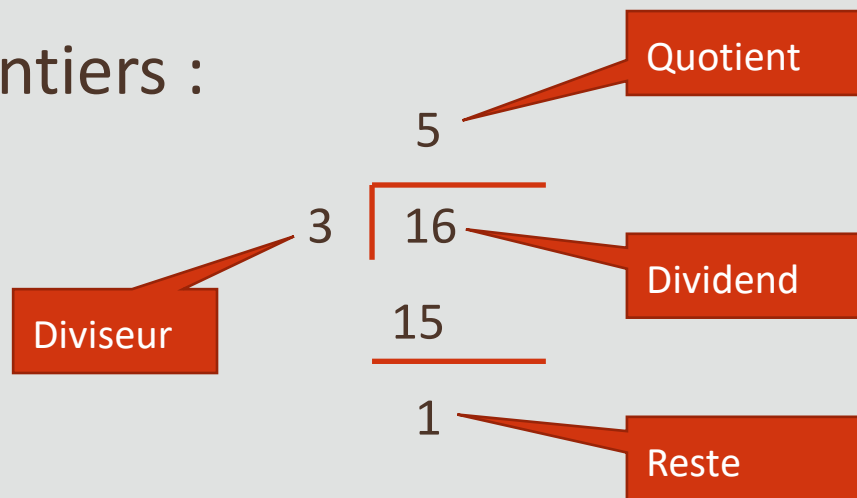
La troncature consiste à supprimer la partie fractionnaire ou décimale d'un nombre. Par exemple : la troncature de 7,8 donne 7 et celle de -3,2 donne -3.



Troncature et division d'entiers

- Exemples de divisions d'entiers :

- $16 / 3 = 5$ (troncature)
- $10 / 3 = 3$ (troncature)
- $16 \% 3 = 1$
- $12 \% 3 = 0$
- $11 \% 3 = 2$



- Pour calculer un quotient sans troncature (résultat décimal), convertissez le dividende ou le diviseur en nombre décimal.
- Par exemple :
 - $11 / 5,0 = 2,2$ et $11,0 / 5 = 2,2$

Troncature et division d'entiers - Exemple 1

- Qu'affiche le code ci-dessous lorsqu'il est exécuté ?

```
int x = 4, y = 5, z = 10;  
int total = z / (x * y);
```

```
System.out.println("Le total est " + total + ".");
```

- La réponse est 0.
- Pourquoi pas 0,5 ?
- Etant donné que le total est un entier, le système stocke une valeur entière pour le résultat du calcul
- La partie décimale du résultat est tronquée plutôt qu'arrondie, générant la réponse finale de 0

Troncature et division d'entiers - Exemple 2

- Un programmeur peut écrire le code suivant pour créer un programme visant à calculer le volume
- Le programme Java obtiendra le résultat incorrect de 0
- En allant de gauche à droite, le programme divise 1 par 3
- Java considère 1 et 3 comme des entiers littéraux et effectue la division d'entiers, et la valeur 0,33333 est tronquée à 0
- Comment corriger cela ?

```
double height = 4, radius = 10, volume;  
volume = 1 / 3 * 3.14 * radius * radius * height;  
System.out.println("Le volume est " + volume + ".");
```

Présentation des types et des conversions

- Vous pouvez empêcher une formule de tronquer une valeur des manières suivantes :
 - Déplacez la fraction à la fin de sorte que Java utilise toujours une valeur double et un entier, et convertisse implicitement la réponse en valeur double au lieu de la tronquer

```
double volume = 3.14 * radius * radius * height * 1 / 3;
```

- Convertissez l'un des entiers littéraux en valeur double littérale de sorte que Java utilise toujours une valeur double et un entier, puis convertisse implicitement la réponse en valeur double au lieu de la tronquer

```
double volume = 1 / 3.0 * 3.14 * radius * radius * height;
```

Conversions de type implicites

- Dans l'exemple précédent, Java a effectué des conversions de type implicites
- Cette opération survient à chaque fois qu'un petit type de données (comme int) est placé dans un type plus grand (comme double)
- Java se rend compte que les types sont différents et effectue automatiquement la conversion vers le plus grand type
- Cependant, Java n'effectue pas automatiquement la conversion du plus grande (double) au plus petit (int)

Utilisation de la conversion de type

- En utilisant la méthode `random` de la bibliothèque `Math`, vous pouvez générer un nombre aléatoire compris entre 1 et 10
- La méthode `random` génère une valeur double comprise entre 0 et 1 (non inclus)
- Des valeurs comme 0, 0,4567 ou 0,901306 peuvent être générées

```
int number;  
number = Math.random() * 10;  
System.out.println("The random number is " + number + ".");
```

Utilisation de la conversion de type

- En multipliant ces valeurs par 10, puis en tronquant le reste, le résultat obtenu est 0, 4 ou 9
- Toutefois, Java ne permet pas la compilation de ce programme dans son état actuel
- Lors d'une conversion d'une valeur supérieure (double) à une valeur inférieure (int), des données sont perdues
- La conversion de type est donc nécessaire

```
int number;  
number = Math.random() * 10;  
System.out.println("The random number is " + number + ".");
```

Opérateur de conversion de type

- Pour convertir une valeur double en int, utilisez (int) devant la valeur
- Pour placer le résultat double de la formule dans le conteneur d'entier, utilisez l'opérateur de conversion de type (int) devant la valeur
- La conversion vers le type de données int tronque la valeur
- Ainsi, la conversion du littéral double 4,567 en valeur de type int donne 4, et celle de 9,01306 donne 9

```
int number;  
number = (int) (Math.random() * 10);  
System.out.println("The random number is " + number + ".");
```

Conversion des types de données

- Vous pouvez convertir un type de données (primitif ou de référence) en un autre type de données en plaçant simplement le nom du type de données entre parenthèses devant la valeur ou la variable, comme indiqué ci-dessous

```
int number;  
Object o;  
char firstInitial = 'A';  
number = (int)firstInitial;  
o = (Object)firstInitial;
```

Conversion des types de données String

- La conversion de type ne fonctionne pas dans toutes les situations
- Par exemple
 - convertir une valeur char au type String entraîne une erreur du compilateur
 - Dans de telles situations, vous devez effectuer la conversion de type d'une autre manière
 - Certaines méthodes de la bibliothèque `java.lang` permettent de convertir les caractères en chaînes

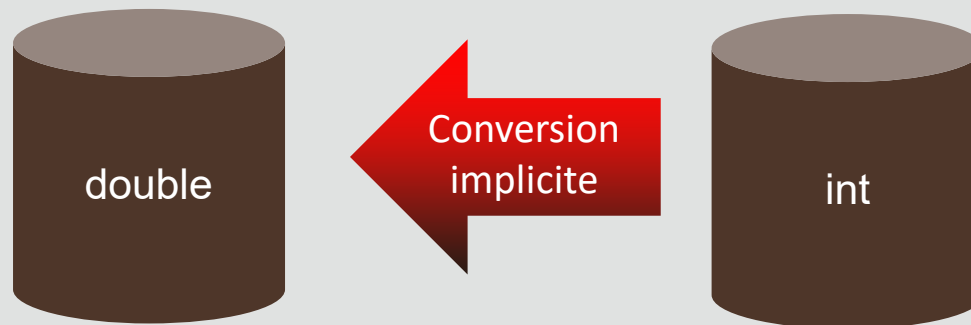
Utilisation des conversions de type

- La conversion de type est une alternative à la résolution du problème de troncature avec la formule de volume donnée précédemment
- Utilisez la conversion de type pour transformer l'un des entiers littéraux en valeur double

```
double volume = (double) 1 / 3 * 3.14 * radius * radius * height;
```

Présentation des types et des conversions

- Lorsque Java effectue la conversion d'un type primitif vers un type primitif plus grand, la conversion est implicite

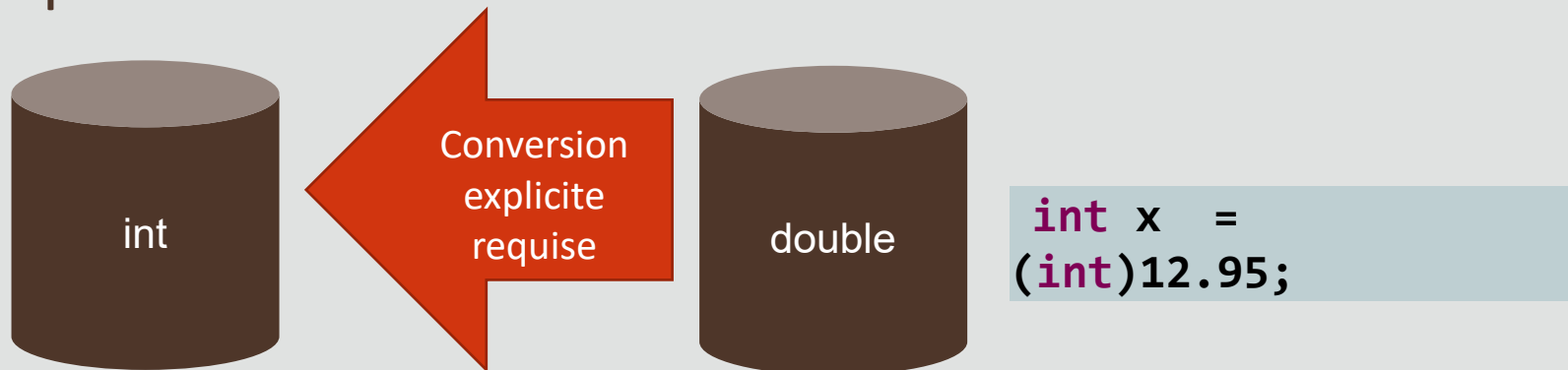


```
double d1 = 5;
```



Présentation des types et des conversions

- Toutefois, lorsque Java effectue une conversion d'un type primitif plus grand à un plus petit, celle-ci doit être explicite
- Java n'effectue pas de conversion implicite d'un type plus grand vers un type plus petit en raison de la perte de précision



Recherche dans l'API Java

- Certains exemples et exercices de ce cours exigent l'utilisation des méthodes dans les classes Java String et Math
- Vous pouvez trouver la description de toutes les méthodes Java dans l'API Java en ligne
- Savoir naviguer dans cette vaste bibliothèque de classes et de méthodes standard vous aidera à écrire des programmes Java, ainsi qu'à réutiliser des blocs de code qui ont été créés par d'autres utilisateurs

Pourquoi utiliser l'API Java ?

- L'un des principaux avantages de l'accès à l'API Java est la réutilisation de code, un concept courant chez les programmeurs
- Pour éviter tout codage excessif, utilisez l'API afin d'accéder à du code existant correspondant exactement à vos besoins
- Vous gagnerez du temps en évitant de reproduire du code existant et optimiserez votre programmation

Consulter l'API Java

- Revenez à l'API Java à l'aide d'un moteur de recherche
- Il existe de nombreuses éditions
- Consultez l'édition Standard Edition pour Java 18 :
<https://docs.oracle.com/en/java/javase/18/docs/api/all/classes-index.html>
 - Examinez la classe Math
 - Regardez si vous pouvez trouver la valeur de pi, ainsi qu'une méthode de calcul de racine carrée

Classe Math

- Trouvez la classe Math dans la fenêtre de cadre de la classe

OVERVIEW MODULE PACKAGE **CLASS** USE TREE PREVIEW NEW DEPRECATED INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH:

1.0

Field Summary

Modifier and Type	Field	Description
static final double	E	The double value that is closer than any other to e , the base of the natural logarithms.
static final double	PI	The double value that is closer than any other to π , the ratio of the circumference of a circle to its diameter.

Method Summary

Modifier and Type	Method	Description
static double	abs(double a)	Returns the absolute value of a double value.
static float	abs(float a)	Returns the absolute value of a float value.
static int	abs(int a)	Returns the absolute value of an int value.
static long	abs(long a)	Returns the absolute value of a long value.
static int	absExact(int a)	Returns the mathematical absolute value of an int value if it is exactly representable as an int, throwing ArithmeticException if the result overflows the positive int range.
static long	absExact(long a)	Returns the mathematical absolute value of a long value if it is exactly representable as a long, throwing ArithmeticException if the result overflows the positive long range.
static double	acos(double a)	Returns the arc cosine of a value; the returned angle is in the range 0.0 through π .
static int	addExact(int x, int y)	Returns the sum of its arguments, throwing an exception if the result overflows an int.
static long	addExact(long x, long y)	Returns the sum of its arguments, throwing an exception if the result overflows a long.
static double	asin(double a)	Returns the arc sine of a value; the returned angle is in the range $-\pi/2$ through $\pi/2$.
static double	atan(double a)	Returns the arc tangent of a value; the returned angle is in the range $-\pi/2$ through $\pi/2$.
static double	atan2(double y, double x)	Returns the angle θ from the conversion of rectangular coordinates (x, y) to polar coordinates (r, θ).
static double	cbrt(double a)	Returns the cube root of a double value.

Faites défiler pour voir la liste des champs et méthodes disponibles dans cette classe

Champ PI

- Faites défiler pour trouver le champ PI
- Pour intégrer PI à un programme, indiquez le nom de la classe (Math) et PI, séparés par l'opérateur point
- Par exemple, ce champ permet d'obtenir un calcul de volume plus précis dans l'exemple précédent s'il est utilisé comme suit :

```
double volume = (double) 1 / 3 * Math.PI * radius * radius * height;
```

Field Summary

Fields

Modifier and Type	Field	Description
static final double	E	The double value that is closer than any other to <i>e</i> , the base of the natural logarithms.
static final double	PI	The double value that is closer than any other to <i>pi</i> , the ratio of the circumference of a circle to its diameter.

Champ PI avec une description

Calcul de racines carrées

- Méthode de calcul des racines carrées :

<code>static double</code>	<code>sqrt(double a)</code> Returns the correctly rounded positive square root of a double value.
----------------------------	--

- Pour calculer la racine carrée de 625, utilisez le nom de la classe et la méthode, séparés à nouveau par l'opérateur point

```
double answer = Math.sqrt(625);
```

- La méthode sqrt exige une valeur double
- Pourquoi l'entier littéral 625 n'entraîne-t-il pas d'erreur ?

Calcul de racines carrées - Solution

- Question :
 - La méthode `sqrt` exige une valeur double
 - Pourquoi l'entier littéral 625 n'entraîne-t-il pas d'erreur ?
- Solution :
 - conversion implicite
 - La valeur 625 de type `int` est convertie en type double de manière implicite, c'est pourquoi aucune erreur n'est générée

Opérateurs et types de données - Exercice

- Sur une feuille, évaluez les instructions Java suivantes et notez leurs résultats

```
double x = 3.25;
double y = -4.5;
int m = 23;
int n = 9;
System.out.println(x + m * y - (y + n) * x);
System.out.println(m / n + m % n);
System.out.println(5 * x - n / 5);
System.out.println(Math.sqrt(Math.sqrt(n)));
System.out.println((int)x);
System.out.println(Math.round(y));
double x = 3.25;
double y = -4.5;
int m = 23;
int n = 9;
System.out.println((int)Math.round(x) + (int)Math.round(y));
System.out.println(m + n);
System.out.println(1-1-((1-(1-(1-n)))));
```

Terminologie

- Termes clés utilisés dans cette leçon :
 - Opérateur arithmétique
 - Opérateur d'affectation
 - boolean
 - char
 - Conventions
 - Déclaration
 - Double
 - float
 - Initialisation
 - int
 - Littéraux

Terminologie

- Termes clés utilisés dans cette leçon :
 - long
 - Ordre des opérations
 - Types de données primitifs
 - Portée
 - short
 - Troncature
 - Conversion (cast) de type
 - Conversion de type
 - Variables

Synthèse

- Dans cette leçon, vous avez appris comment :
 - Utiliser les types de données primitifs dans le code Java
 - Spécifier des littéraux pour les types primitifs et pour les chaînes
 - Montrer l'initialisation des variables
 - Décrire les règles de portée d'une méthode
 - Déterminer si une expression exige une conversion de type
 - Appliquer la conversion (cast) de type dans le code Java
 - Utiliser les opérateurs arithmétiques
 - Utiliser l'opérateur d'affectation
 - Utiliser une méthode de la classe Math
 - Accéder à une méthode de la classe Math à partir de l'API Java





ORACLE

Academy

